

PLAN DE DÉVELOPPEMENT & TO-DO LIST TECHNIQUE

Application de gestion - Cité Étoile du Monde

Technologie	Laravel + Blade + MySQL
Architecture	Application web monolithique modulaire
Priorités	Sécurité forte - Tests unitaires - Traçabilité
Version	1.0 - Septembre 2026

Document de pilotage du développement logiciel

1. Objet du document

Ce document transforme le cahier des charges fonctionnel validé et le schéma de base de données validé en un plan de développement concret, séquencé et contrôlable.

La plateforme sera développée avec Laravel côté serveur, Blade pour l'interface utilisateur et MySQL comme système de gestion de base de données. Le projet doit être sécurisé par conception et couvert par des tests automatisés dès les premières phases.

La To-do list ne doit pas être lue comme une simple liste de tickets. Elle constitue la feuille de route de réalisation du produit, avec dépendances, critères d'acceptation, contrôles de sécurité et stratégie de tests.

2. Principes directeurs

Sécurité par défaut

Toute fonctionnalité sensible doit être protégée par authentification, autorisation, validation, journalisation et tests.

Tests dès la conception

Chaque module métier doit inclure ses tests unitaires et fonctionnels avant d'être considéré comme terminé.

Traçabilité

Les paiements, reçus, souscriptions et actions administratives sensibles doivent être auditable.

Intégrité financière

Aucun paiement validé ne doit être supprimé silencieusement. Les corrections passent par annulation ou reversal contrôlé.

Séparation des responsabilités

Les rôles Administrateur, Commercial, Caissier, Finance, Direction et Client ne disposent pas des mêmes permissions.

Portail client sécurisé

Un client ne doit jamais pouvoir accéder aux données d'un autre client, même en modifiant une URL ou un identifiant.

Évolutivité

L'application doit pouvoir évoluer vers la gestion résidentielle de la cité sans remettre en cause le noyau commercial et financier.

3. Stack technique retenue

Couche	Technologie	Rôle
Backend	Laravel	Logique métier, sécurité, API interne, génération de documents
Frontend	Blade + JavaScript léger	Interfaces Admin et Client
Base de données	MySQL 8.x / InnoDB	Données métier et financières
CSS	CSS structuré ou framework UI validé ultérieurement	Design responsive et cohérent
Tests	PHPUnit / Pest selon standard projet	Tests unitaires et fonctionnels
PDF	Librairie Laravel dédiée	Reçus et rapports PDF
Logs	Laravel logging + audit métier	Traçabilité technique et fonctionnelle

4. Découpage fonctionnel de la plateforme

01. Authentification et sécurité

02. Utilisateurs, rôles et permissions

03. Clients

04. Quartiers, avenues et parcelles

05. Formules d'acquisition

06. Souscriptions et contrats

07. Échéanciers

08. Paiements

09. Affectation des paiements

10. Reçus PDF

11. Dashboard administratif

12. Rapports et statistiques

13. Logs et audit

14. Settings

15. Portail client

16. Documents et pièces jointes

5. Exigences de sécurité transversales

Authentification : Laravel Authentication avec mots de passe hashés, rotation de session après connexion, expiration de session, limitation des tentatives de connexion.

Autorisation : Policies/Gates Laravel et permissions par rôle. Aucune autorisation uniquement basée sur l'affichage de boutons.

CSRF : Protection CSRF obligatoire sur tous les formulaires Blade.

XSS : Échapper les sorties Blade par défaut. Toute sortie HTML non échappée doit être justifiée.

SQL Injection : Utiliser Eloquent ou Query Builder avec paramètres liés. Aucun SQL dynamique non maîtrisé.

Mass assignment : Définir explicitement fillable/guarded et valider les DTO/Form Requests.

IDOR : Vérifier systématiquement que l'utilisateur possède ou a le droit d'accéder à la ressource demandée.

Paielements : Transactions DB, verrouillage si nécessaire, idempotence, références uniques, interdiction de suppression silencieuse.

Fichiers : Validation MIME, extension, taille, stockage non public pour documents sensibles, URL signées pour téléchargement.

Audit : Journaliser les opérations sensibles avec utilisateur, entité, ancienne valeur, nouvelle valeur, date, IP et user-agent lorsque disponible.

Secrets : Aucun secret dans Git. Utiliser .env et variables d'environnement.

HTTPS : Obligatoire en production, cookies Secure/HttpOnly/SameSite adaptés.

2FA : À prévoir au minimum pour les comptes administratifs sensibles.

Sauvegardes : Backups chiffrés, test de restauration, politique de rétention.

Dépendances : Audit régulier des dépendances Composer/NPM et mises à jour de sécurité.

6. Stratégie de tests

Le projet doit adopter une politique de tests automatisés systématique. Une tâche métier n'est pas terminée tant que les scénarios critiques associés ne sont pas couverts.

Les tests unitaires doivent couvrir les règles métier isolées. Les Feature Tests Laravel doivent couvrir les flux HTTP, les autorisations, les validations, les transactions et les réponses utilisateur.

Type de test	Cible	Exemples
--------------	-------	----------

Unitaires	Services métier, calculateurs, règles	Calcul solde, répartition paiement, génération échéancier
Feature	Routes, contrôleurs, policies	Connexion, création client, paiement, téléchargement reçu
Database	Contraintes et transactions	Une parcelle active unique, rollback paiement
Security	Contrôles d'accès	IDOR, accès d'un client aux données d'un autre
Regression	Flux critiques	Souscription -> échéancier -> paiement -> reçu

Objectif recommandé : couverture forte sur le noyau financier et de sécurité, avec priorité absolue aux règles métier critiques plutôt qu'à un pourcentage global artificiel.

PHASE 0 - Initialisation du projet

Objectif : Créer une base technique propre, versionnée et reproductible.

ID	Tâche	Priorité	Critère d'acceptation	Tests / Sécurité
P0-01	Créer le projet Laravel et configurer MySQL	Critique	Application démarre en local et se connecte à MySQL	Test de connexion DB ; .env.example sans secret
P0-02	Configurer Git, branches et conventions de commit	Haute	Dépôt propre avec README et règles de contribution	Aucun secret dans Git
P0-03	Configurer environnements local/test/production	Critique	Configurations séparées et documentées	APP_DEBUG=false en production
P0-04	Configurer PHPUnit/Pest et base de test	Critique	Tests exécutables via une seule commande	Base dédiée aux tests
P0-05	Configurer analyse statique / formatage	Moyenne	Code style cohérent	CI bloque les erreurs critiques
P0-06	Configurer journalisation technique	Haute	Logs applicatifs exploitables	Aucune donnée sensible brute dans logs

Livrable de phase : Projet Laravel initialisé, MySQL connecté, pipeline de tests opérationnel.

PHASE 1 - Schéma de données et migrations

Objectif : Implémenter la base validée avec contraintes et seeders.

ID	Tâche	Priorité	Critère d'acceptation	Tests / Sécurité
P1-01	Créer migrations users, roles, permissions	Critique	Tables et contraintes créées	Tests migrations
P1-02	Créer migrations customers et customer_documents	Critique	Relations correctes	Tests FK et unicité
P1-03	Créer neighborhoods, avenues, plots	Critique	Quartier -> Avenue -> Parcelle fonctionnel	Tests relations et statuts
P1-04	Créer payment_plans	Critique	5 formules seedées	Tests montants/durées
P1-05	Créer subscriptions et contracts	Critique	Snapshot financier présent	Test parcelle active unique
P1-06	Créer installments	Critique	Échéancier persistant	Contraintes montant/solde
P1-07	Créer payments et payment_allocations	Critique	Paielements partiels/anticipés modélisés	Transactions DB
P1-08	Créer receipts, audit_logs, settings	Haute	Tables disponibles	Tests références uniques
P1-09	Créer factories et seeders	Haute	Données de démonstration reproductibles	Pas de données sensibles réelles

Livrable de phase : Migrations, seeders, factories et tests de base de données validés.

PHASE 2 - Authentification, rôles et permissions

Objectif : Sécuriser l'accès à la plateforme dès le début.

ID	Tâche	Priorité	Critère d'acceptation	Tests / Sécurité
P2-01	Implémenter login/logout/reset mot de passe	Critique	Flux complet opérationnel	Rate limiting, rotation session
P2-02	Créer rôles initiaux	Critique	super_admin, admin, commercial, cashier, finance_manager, direction, customer	Tests d'affectation
P2-03	Créer matrice de permissions	Critique	Permissions explicites par action	Tests interdictions
P2-04	Implémenter Policies/Gates	Critique	Chaque ressource protégée serveur	Tests 403
P2-05	Ajouter 2FA administrateurs	Haute	2FA activable pour comptes sensibles	Tests activation/récupération
P2-06	Durcir sessions/cookies	Critique	Cookies sécurisés en production	Secure/HttpOnly/SameSite

Livrable de phase : Authentification sécurisée et RBAC complet.

PHASE 3 - Back-office - Clients

Objectif : Gérer le dossier client complet.

ID	Tâche	Priorité	Critère d'acceptation	Tests / Sécurité
P3-01	Liste clients avec recherche et filtres	Haute	Recherche nom/téléphone/n° client	Autorisation par rôle
P3-02	Création client	Critique	Validation serveur complète	Tests Form Request
P3-03	Fiche client 360°	Critique	Profil, parcelles, souscriptions, paiements, reçus visibles	Pas de fuite inter-client
P3-04	Modification client	Haute	Champs autorisés modifiables	Audit ancienne/nouvelle valeur
P3-05	Documents client	Haute	Upload/téléchargement sécurisé	Validation MIME, stockage privé
P3-06	Archivage client	Moyenne	Pas de suppression destructive	Journalisation

Livrable de phase : Module Clients complet et audité.

PHASE 4 - Quartiers, avenues et parcelles

Objectif : Administrer le patrimoine foncier commercialisable.

ID	Tâche	Priorité	Critère d'acceptation	Tests / Sécurité
P4-01	CRUD quartiers	Haute	Quartiers configurables	Permissions admin
P4-02	CRUD avenues	Haute	Avenue liée à un quartier	Tests FK
P4-03	CRUD parcelles	Critique	Référence unique et statuts séparés	Tests unicité
P4-04	Filtres parcelles	Haute	Filtre quartier/avenue/statut	Tests requêtes
P4-05	Fiche parcelle	Critique	Historique et situation visible	Accès contrôlé
P4-06	Prévenir double attribution	Critique	Impossible d'avoir 2 souscriptions actives	Test concurrence/contrainte DB

Livrable de phase : Module foncier opérationnel.

PHASE 5 - Formules d'acquisition

Objectif : Gérer les conditions commerciales sans casser les contrats existants.

ID	Tâche	Priorité	Critère d'acceptation	Tests / Sécurité
P5-01	Lister et afficher les 5 formules	Haute	Données conformes à la base validée	Tests valeurs
P5-02	Créer/modifier/désactiver une formule	Haute	Historique contractuel préservé	Permissions restreintes
P5-03	Empêcher modification rétroactive	Critique	Souscription garde son snapshot	Test de régression
P5-04	Afficher périodes de validité	Moyenne	valid_from/valid_until respectés	Tests dates

Livrable de phase : Module Formules stable et historisable.

PHASE 6 - Souscriptions et contrats

Objectif : Créer le cœur commercial reliant client, parcelle et formule.

ID	Tâche	Priorité	Critère d'acceptation	Tests / Sécurité
P6-01	Créer une souscription	Critique	Client + parcelle + formule requis	Transaction DB
P6-02	Copier snapshot financier	Critique	Prix/mensualité/durée figés	Test modification formule ultérieure
P6-03	Changer statuts souscription	Haute	Transitions contrôlées	Policies + audit
P6-04	Créer dossier contrat	Haute	N° contrat unique et document lié	Upload privé
P6-05	Vue détail souscription	Critique	Situation complète visible	Autorisation
P6-06	Journaliser opérations sensibles	Critique	Audit systématique	Tests audit_logs

Livrable de phase : Flux de souscription complet et sécurisé.

PHASE 7 - Génération des échéanciers

Objectif : Automatiser les échéances selon la formule souscrite.

ID	Tâche	Priorité	Critère d'acceptation	Tests / Sécurité
P7-01	Créer service InstallmentScheduleService	Critique	Génère N échéances exactes	Tests unitaires
P7-02	Gérer formule cash	Critique	Aucune échéance mensuelle incohérente	Tests scénario cash
P7-03	Gérer 12/36/60/120 mois	Critique	Dates et montants corrects	Tests paramétrés
P7-04	Calcul statuts échéances	Haute	upcoming/due/partial/paid/overdue cohérents	Tests dates
P7-05	Vue échéancier admin	Haute	Lecture claire du calendrier	Accès protégé

Livrable de phase : Moteur d'échéancier couvert par tests unitaires.

PHASE 8 - Paiements et allocation

Objectif : Implémenter un noyau financier robuste.

ID	Tâche	Priorité	Critère d'acceptation	Tests / Sécurité
P8-01	Créer PaymentService transactionnel	Critique	Paiement enregistré atomiquement	DB transaction + rollback
P8-02	Paiement partiel	Critique	Allocation partielle correcte	Tests unitaires
P8-03	Paiement multi-échéances	Critique	Allocation ancienne échéance d'abord	Tests unitaires
P8-04	Paiement anticipé	Critique	Échéances futures affectables	Tests fonctionnels
P8-05	Recalcul souscription	Critique	amount_paid et balance cohérents	Test invariants
P8-06	Mettre à jour statuts financiers	Critique	Statuts calculés correctement	Tests
P8-07	Annulation/reversal contrôlé	Critique	Jamais de DELETE financier	Permissions + audit
P8-08	Idempotence paiement	Haute	Double soumission ne crée pas 2 paiements	Clé/référence unique + test
P8-09	Gestion concurrence	Haute	Deux caissiers ne corrompent pas solde	Lock/transaction + test

Livrable de phase : Module Paiements sécurisé, transactionnel et fortement testé.

PHASE 9 - Reçus PDF

Objectif : Générer une preuve de paiement fiable et vérifiable.

ID	Tâche	Priorité	Critère d'acceptation	Tests / Sécurité
P9-01	Créer modèle reçu	Haute	Numéro, client, parcelle, montant, solde, agent	Tests données
P9-02	Générer PDF	Critique	PDF lisible et téléchargeable	Test génération
P9-03	Créer code de vérification	Haute	Code unique par reçu	Unicité DB
P9-04	Ajouter QR code si retenu	Moyenne	QR vérifie le reçu	URL non sensible
P9-05	Réimpression contrôlée	Haute	Même reçu, pas de duplication	Audit
P9-06	Annulation reçu liée au paiement	Critique	Cohérence avec reversal	Tests régression

Livrable de phase : Reçus PDF fiables, immuables après validation et téléchargeables.

PHASE 10 - Dashboard administratif

Objectif : Donner une lecture immédiate de la situation commerciale et financière.

ID	Tâche	Priorité	Critère d'acceptation	Tests / Sécurité
P10-01	KPI clients/parcelles	Haute	Totaux cohérents avec DB	Tests agrégats
P10-02	KPI financiers	Critique	encaissé, restant, retards corrects	Tests calculs
P10-03	Graphiques paiements	Moyenne	jour/semaine/mois/année	Filtres validés
P10-04	Répartition formules	Moyenne	Statistiques exactes	Tests agrégats
P10-05	Échéances proches/retards	Haute	Listes actionnables	Accès finance/direction

Livrable de phase : Dashboard directionnel cohérent avec les données transactionnelles.

PHASE 11 - Portail Client

Objectif : Permettre au client de consulter uniquement son propre dossier.

ID	Tâche	Priorité	Critère d'acceptation	Tests / Sécurité
P11-01	Compte client lié à customer	Critique	Relation 1:1 contrôlée	Tests ownership
P11-02	Dashboard client	Critique	Parcelle, formule, payé, reste, prochaine échéance	Feature tests
P11-03	Mes souscriptions	Haute	Toutes ses parcelles visibles	IDOR tests
P11-04	Mes paiements	Critique	Historique exact	IDOR tests
P11-05	Mes reçus	Critique	Voir/télécharger uniquement ses reçus	Signed route + policy
P11-06	Mon échéancier	Haute	Statut de chaque échéance	Tests
P11-07	Mes informations personnelles	Haute	Lecture et modifications autorisées	Validation + audit
P11-08	Sécurité anti-IDOR	Critique	Changer un ID ne donne jamais accès à autrui	Tests dédiés

Livrable de phase : Portail client sécurisé et autonome en consultation.

PHASE 12 - Rapports et exports

Objectif : Fournir les outils de pilotage et de contrôle.

ID	Tâche	Priorité	Critère d'acceptation	Tests / Sécurité
P12-01	Rapports clients	Haute	Filtres période/statut	Tests filtres
P12-02	Rapports parcelles	Haute	Disponibles/vendues/ réservées	Tests agrégats
P12-03	Rapports paiements	Critique	Par période, formule, agent	Accès finance
P12-04	Rapport impayés	Critique	Montant et ancienneté corrects	Tests dates
P12-05	Situation client PDF	Haute	Historique + solde + échancier	Test génération
P12-06	Export CSV/Excel si retenu	Moyenne	Export conforme aux filtres	Contrôle permissions

Livrable de phase : Module de reporting exploitable par Finance et Direction.

PHASE 13 - Audit et logs

Objectif : Rendre les actions sensibles traçables.

ID	Tâche	Priorité	Critère d'acceptation	Tests / Sécurité
P13-01	Créer AuditService	Critique	API uniforme de journalisation	Tests unitaires
P13-02	Journaliser paiements/reversals	Critique	Avant/après disponible	Tests
P13-03	Journaliser souscriptions	Haute	Transitions historisées	Tests
P13-04	Journaliser settings et permissions	Critique	Toute modification sensible tracée	Tests
P13-05	Écran de consultation logs	Haute	Recherche utilisateur/action/date	Accès super_admin/direction
P13-06	Protéger logs contre édition	Critique	Aucun CRUD standard de modification	Policies + DB

Livrable de phase : Journal d'audit complet pour opérations critiques.

PHASE 14 - Settings

Objectif : Centraliser les paramètres configurables.

ID	Tâche	Priorité	Critère d'acceptation	Tests / Sécurité
P14-01	Paramètres entreprise/projet	Haute	Nom, logo, contacts configurables	Permissions
P14-02	Paramètres finance	Haute	Devise, méthodes de paiement	Validation
P14-03	Préfixes de numérotation	Moyenne	Client, paiement, reçu, contrat	Tests unicité
P14-04	Règles paiement partiel/anticipé	Haute	Valeurs reflétées dans services	Tests configuration
P14-05	Journaliser toute modification	Critique	Audit automatique	Tests

Livrable de phase : Paramètres administrables sans modifier le code.

PHASE 15 - UI/UX Blade

Objectif : Construire une interface cohérente, responsive et sûre.

ID	Tâche	Priorité	Critère d'acceptation	Tests / Sécurité
P15-01	Créer layout Blade global	Haute	Sidebar/header/footer cohérents	Aucune donnée sensible en JS
P15-02	Composants formulaires	Haute	Erreurs et old values gérés	CSRF partout
P15-03	Tables et filtres	Haute	Pagination/tri/recherche	Validation query params
P15-04	Responsive desktop/tablette/mobile	Haute	Flux critiques utilisables mobile	Tests visuels
P15-05	États loading/empty/error	Moyenne	UX claire	Pas d'exception brute
P15-06	Accessibilité de base	Moyenne	Labels, focus, contraste, clavier	Audit manuel

Livrable de phase : Interface Blade professionnelle et cohérente.

PHASE 16 - Sécurité renforcée et durcissement

Objectif : Effectuer une revue de sécurité dédiée avant mise en production.

ID	Tâche	Priorité	Critère d'acceptation	Tests / Sécurité
P16-01	Revue OWASP des flux critiques	Critique	Checklist documentée	Auth, IDOR, XSS, CSRF, injection
P16-02	Tester autorisations par rôle	Critique	Aucune élévation de privilège	Suite automatisée
P16-03	Tester portail client anti-IDOR	Critique	Isolation complète des clients	Tests négatifs
P16-04	Rate limiting endpoints sensibles	Haute	Login/reset protégés	Tests limites
P16-05	Durcir upload fichiers	Critique	MIME/taille/nom/stockage sécurisé	Tests fichiers malveillants
P16-06	Vérifier headers sécurité	Haute	Headers adaptés	Contrôle production
P16-07	Audit dépendances	Haute	Pas de vulnérabilité critique connue	composer audit/npm audit
P16-08	Revue logs et données sensibles	Haute	Pas de mots de passe/tokens en logs	Inspection

Livrable de phase : Rapport de sécurité pré-production avec anomalies critiques corrigées.

PHASE 17 - Tests finaux et recette

Objectif : Valider l'application avant déploiement.

ID	Tâche	Priorité	Critère d'acceptation	Tests / Sécurité
P17-01	Exécuter suite complète	Critique	0 test critique en échec	CI verte
P17-02	Tests du workflow complet	Critique	Client -> souscription -> paiement -> reçu	E2E fonctionnel
P17-03	Tests cash et crédits	Critique	5 formules validées	Cas limites
P17-04	Tests paiements partiels/anticipés	Critique	Allocation correcte	Régression
P17-05	Tests annulation/reversal	Critique	Solde restauré correctement	Transactions
P17-06	Recette par rôles	Haute	Chaque profil valide ses écrans	Checklist métier
P17-07	Test sauvegarde/restauration	Critique	Restauration réellement réussie	Procédure documentée

Livrable de phase : Version candidate validée pour production.

PHASE 18 - Déploiement et exploitation

Objectif : Mettre en production de façon contrôlée.

ID	Tâche	Priorité	Critère d'acceptation	Tests / Sécurité
P18-01	Préparer serveur production	Critique	HTTPS, PHP, MySQL, backups	Hardening serveur
P18-02	Déployer application	Critique	Migrations et cache réussis	Maintenance mode si besoin
P18-03	Configurer queue/cron	Haute	Tâches planifiées actives	Monitoring
P18-04	Configurer sauvegardes	Critique	Backups automatiques testés	Chiffrement/rétention
P18-05	Configurer monitoring/logs	Haute	Erreurs détectables	Alertes
P18-06	Créer super admin initial	Critique	Compte sécurisé + 2FA	Mot de passe temporaire changé
P18-07	Former utilisateurs clés	Haute	Guide et démonstration	Validation métier

Livrable de phase : Application en production, supervisée et sauvegardée.

7. Jalons recommandés

Jalon	Contenu	Condition de validation
M1 - Fondations	Phases 0 à 2	Projet, DB, auth et RBAC stables
M2 - Noyau métier	Phases 3 à 7	Clients, foncier, souscriptions et échéanciers
M3 - Finance	Phases 8 à 9	Paielements et reçus sécurisés
M4 - Pilotage	Phases 10, 12, 13, 14	Dashboard, rapports, audit, settings
M5 - Portail client	Phase 11	Client autonome et isolé
M6 - Production	Phases 15 à 18	UX, sécurité, tests, déploiement

8. Définition de “Done”

- Le code est relu et respecte les conventions du projet.
- La fonctionnalité est couverte par les tests unitaires/Feature pertinents.
- Les permissions sont vérifiées côté serveur.
- Les entrées utilisateur sont validées par Form Request ou mécanisme équivalent.
- Les opérations sensibles sont journalisées.
- Les migrations/seeders nécessaires sont inclus.
- Aucune erreur critique n'apparaît dans les logs.
- La fonctionnalité fonctionne sur desktop et mobile lorsque pertinent.
- Les scénarios d'échec sont gérés proprement.
- La documentation technique minimale est mise à jour.

9. Matrice de tests critiques

Scénario	Résultat attendu	Type	Criticité
Client A tente d'ouvrir le reçu de Client B	403/404 sans fuite d'information	Feature/Security	Critique
Deux souscriptions actives	Deuxième création refusée	DB/Feature	Critique

pour la même parcelle			
Paieement 100 sur échéance 175	Échéance partiellement payée, reste 75	Unit/Feature	Critique
Paieement 500 sur mensualité 175	175 + 175 + 150 correctement alloués	Unit	Critique
Double clic sur validation paieement	Un seul paieement créé	Feature/DB	Critique
Erreur pendant génération paieement	Transaction rollback complet	Feature/DB	Critique
Modification formule après souscription	Ancien contrat inchangé	Regression	Critique
Commercial tente annuler paieement	Refus autorisation	Security	Critique
Fichier exécutable uploadé comme document	Upload refusé	Security	Haute
Paieement soldant la souscription	Statut financier devient paid	Unit/Feature	Critique

10. Backlog hors V1

- Notifications WhatsApp/SMS automatiques
- Paiement en ligne
- Plan interactif de la cité
- Application mobile dédiée
- Gestion des résidents et visiteurs
- Gestion de maintenance et équipements
- Portail fournisseur/prestataire
- BI avancée et prévisions financières

11. Conclusion

La réalisation doit privilégier la fiabilité du noyau financier, la sécurité des accès et la traçabilité avant les fonctionnalités esthétiques ou secondaires. Le chemin critique du projet est : authentification et données -> clients/parcelles -> souscriptions -> échéanciers -> paiements -> reçus -> portail client -> reporting.

La To-do list ci-dessus doit servir de référence pour créer les tickets de développement. Chaque phase peut ensuite être découpée en issues GitHub ou tâches de sprint sans modifier l'ordre des dépendances principales.

12. Index rapide des phases

Phase 0 - Initialisation

Phase 1 - Base de données

Phase 2 - Authentification/RBAC

Phase 3 - Clients

Phase 4 - Quartiers/Avenues/Parcelles

Phase 5 - Formules

Phase 6 - Souscriptions/Contrats

Phase 7 - Échéanciers

Phase 8 - Paiements

Phase 9 - Reçus PDF

Phase 10 - Dashboard Admin

Phase 11 - Portail Client

Phase 12 - Rapports

Phase 13 - Audit/Logs

Phase 14 - Settings

Phase 15 - UI/UX Blade

Phase 16 - Sécurité renforcée

Phase 17 - Tests/Recette

Phase 18 - Déploiement

13. Annexe A - Base de données MySQL validée

La base de données retenue pour la plateforme est MySQL 8.x avec le moteur InnoDB, l'encodage utf8mb4 et des clés étrangères pour garantir l'intégrité référentielle. Le script ci-dessous constitue le schéma SQL de référence de la V1. Les migrations Laravel devront reproduire ces contraintes sans dépendre d'un import SQL manuel en production.

Principes structurants : un client peut avoir plusieurs souscriptions/parcelles ; une parcelle ne peut avoir qu'une souscription active à la fois ; les paiements partiels et anticipés sont autorisés ; l'organisation géographique suit Quartier -> Avenue -> Parcelle ; les montants contractuels sont figés dans la souscription.

Bloc	Tables principales	Responsabilité
Sécurité	users, roles, permissions, user_roles, role_permissions	Authentification et contrôle d'accès
Clients	customers, customer_documents	Dossier client et documents
Foncier	neighborhoods, avenues, plots	Quartiers, avenues et parcelles
Commercial	payment_plans, subscriptions, contracts	Formules et souscriptions

Finance	installments, payments, payment_allocations, receipts	Échéances, encaissements et reçus
Administration	audit_logs, settings	Traçabilité et paramètres

13.1 Contraintes de données essentielles

- Un client peut être lié à plusieurs souscriptions.
- Une souscription concerne exactement une parcelle et une formule de paiement.
- Une parcelle ne peut avoir qu'une souscription pending/active/suspended à la fois ; MySQL applique cette règle via une colonne générée et un index UNIQUE.
- Le snapshot contractuel (prix total, mensualité, durée) est conservé dans subscriptions afin qu'une modification future d'une formule n'altère pas un contrat existant.
- Les paiements partiels et anticipés sont ventilés par payment_allocations vers une ou plusieurs échéances.
- Les soldes de souscription et d'échéance sont des colonnes générées dans MySQL pour limiter les incohérences arithmétiques.
- Les suppressions destructives sont évitées pour les données financières ; Laravel doit utiliser des statuts d'annulation/reversal et journaliser l'opération.
- Les documents sensibles doivent être stockés hors du répertoire public et servis via des contrôles d'autorisation.

13.2 Script SQL MySQL 8.x

Le script complet est également livré séparément sous le nom cite_etoile_du_monde_schema_mysql.sql. Il est reproduit ci-dessous pour que le document de planification reste autonome.

```
-- =====
-- CITE ETOILE DU MONDE - MJIC IMMOBILIER SARL
-- Base de données MySQL 8.x - Version initiale validée
-- Stack cible : Laravel + Blade + MySQL
-- =====

SET NAMES utf8mb4;
SET time_zone = '+00:00';
SET FOREIGN_KEY_CHECKS = 0;

-- Optionnel : créer et sélectionner la base manuellement si nécessaire.
-- CREATE DATABASE cite_etoile_du_monde
-- CHARACTER SET utf8mb4
-- COLLATE utf8mb4_unicode_ci;
-- USE cite_etoile_du_monde;

-- =====
-- 1. USERS / AUTHENTICATION
-- =====

CREATE TABLE users (
  id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  first_name VARCHAR(100) NOT NULL,
  last_name VARCHAR(100) NOT NULL,
  email VARCHAR(190) NOT NULL UNIQUE,
  phone VARCHAR(50) NULL,
  password VARCHAR(255) NOT NULL,
  remember_token VARCHAR(100) NULL,
  status ENUM('active', 'suspended', 'disabled') NOT NULL DEFAULT 'active',
  last_login_at DATETIME NULL,
  created_at TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  INDEX idx_users_status (status)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE roles (
  id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL UNIQUE,
  description TEXT NULL,
  created_at TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

```

CREATE TABLE permissions (
  id BIGINT UNSIGNED AUTO INCREMENT PRIMARY KEY,
  name VARCHAR(150) NOT NULL UNIQUE,
  description TEXT NULL,
  created_at TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE user_roles (
  user_id BIGINT UNSIGNED NOT NULL,
  role_id BIGINT UNSIGNED NOT NULL,
  created_at TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (user_id, role_id),
  CONSTRAINT fk_user_roles_user
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
  CONSTRAINT fk_user_roles_role
    FOREIGN KEY (role_id) REFERENCES roles(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE role_permissions (
  role_id BIGINT UNSIGNED NOT NULL,
  permission_id BIGINT UNSIGNED NOT NULL,
  created_at TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (role_id, permission_id),
  CONSTRAINT fk_role_permissions_role
    FOREIGN KEY (role_id) REFERENCES roles(id) ON DELETE CASCADE,
  CONSTRAINT fk_role_permissions_permission
    FOREIGN KEY (permission_id) REFERENCES permissions(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- =====
-- 2. CLIENTS
-- =====

CREATE TABLE customers (
  id BIGINT UNSIGNED AUTO INCREMENT PRIMARY KEY,
  customer_number VARCHAR(50) NOT NULL UNIQUE,
  user_id BIGINT UNSIGNED NULL UNIQUE,
  first_name VARCHAR(100) NOT NULL,
  last_name VARCHAR(100) NOT NULL,
  middle_name VARCHAR(100) NULL,
  gender VARCHAR(20) NULL,
  birth_date DATE NULL,
  phone VARCHAR(50) NOT NULL,
  whatsapp VARCHAR(50) NULL,
  email VARCHAR(190) NULL,
  address TEXT NULL,
  city VARCHAR(100) NULL,
  country VARCHAR(100) NULL,
  nationality VARCHAR(100) NULL,
  status ENUM('prospect', 'active', 'settled', 'suspended', 'archived') NOT NULL DEFAULT 'active',
  created_by BIGINT UNSIGNED NULL,
  created_at TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  INDEX idx_customers_name (last_name, first_name),
  INDEX idx_customers_phone (phone),
  INDEX idx_customers_status (status),
  CONSTRAINT fk_customers_user
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE SET NULL,
  CONSTRAINT fk_customers_created_by
    FOREIGN KEY (created_by) REFERENCES users(id) ON DELETE SET NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE customer_documents (
  id BIGINT UNSIGNED AUTO INCREMENT PRIMARY KEY,
  customer_id BIGINT UNSIGNED NOT NULL,
  document_type ENUM(
    'identity', 'photo', 'contract', 'payment_proof',
    'subscription_document', 'other'
  ) NOT NULL,
  name VARCHAR(255) NOT NULL,
  file_path TEXT NOT NULL,
  uploaded_by BIGINT UNSIGNED NULL,
  created_at TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP,
  INDEX idx_customer_documents_customer (customer_id),
  CONSTRAINT fk_customer_documents_customer
    FOREIGN KEY (customer_id) REFERENCES customers(id) ON DELETE CASCADE,
  CONSTRAINT fk_customer_documents_uploaded_by
    FOREIGN KEY (uploaded_by) REFERENCES users(id) ON DELETE SET NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- =====
-- 3. ORGANISATION DE LA CITE : QUARTIER -> AVENUE -> PARCELLE
-- =====

CREATE TABLE neighborhoods (
  id BIGINT UNSIGNED AUTO INCREMENT PRIMARY KEY,
  code VARCHAR(50) NOT NULL UNIQUE,
  name VARCHAR(150) NOT NULL UNIQUE,
  description TEXT NULL,
  status ENUM('planned', 'active', 'commercializable', 'completed', 'suspended')
  NOT NULL DEFAULT 'active',
  created_at TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  INDEX idx_neighborhoods_status (status)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE avenues (
  id BIGINT UNSIGNED AUTO INCREMENT PRIMARY KEY,
  neighborhood_id BIGINT UNSIGNED NOT NULL,
  code VARCHAR(50) NOT NULL,
  name VARCHAR(150) NOT NULL,
  description TEXT NULL,

```

```

status ENUM('planned', 'active', 'suspended') NOT NULL DEFAULT 'active',
created_at TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
UNIQUE KEY uq_avenues_neighborhood_code (neighborhood_id, code),
UNIQUE KEY uq_avenues_neighborhood_name (neighborhood_id, name),
INDEX idx_avenues_neighborhood (neighborhood_id),
CONSTRAINT fk_avenues_neighborhood
    FOREIGN KEY (neighborhood_id) REFERENCES neighborhoods(id) ON DELETE RESTRICT
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE plots (
    id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    plot_number VARCHAR(50) NOT NULL,
    reference VARCHAR(100) NOT NULL UNIQUE,
    avenue_id BIGINT UNSIGNED NOT NULL,
    surface_area DECIMAL(12,2) NULL,
    width DECIMAL(10,2) NULL,
    length DECIMAL(10,2) NULL,
    cadastral_reference VARCHAR(150) NULL,
    base_price DECIMAL(12,2) NULL,
    commercial_status ENUM('available', 'reserved', 'subscribed', 'blocked', 'unavailable')
        NOT NULL DEFAULT 'available',
    financial_status ENUM('unpaid', 'partially_paid', 'paid')
        NOT NULL DEFAULT 'unpaid',
    administrative_status ENUM('not_started', 'in_progress', 'validated', 'allocated', 'dispute')
        NOT NULL DEFAULT 'not_started',
    notes TEXT NULL,
    created_at TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    UNIQUE KEY uq_plots_avenue_number (avenue_id, plot_number),
    INDEX idx_plots_avenue (avenue_id),
    INDEX idx_plots_commercial_status (commercial_status),
    INDEX idx_plots_financial_status (financial_status),
    CONSTRAINT chk_plots_surface_area CHECK (surface_area IS NULL OR surface_area > 0),
    CONSTRAINT chk_plots_width CHECK (width IS NULL OR width > 0),
    CONSTRAINT chk_plots_length CHECK (length IS NULL OR length > 0),
    CONSTRAINT chk_plots_base_price CHECK (base_price IS NULL OR base_price >= 0),
    CONSTRAINT fk_plots_avenue
        FOREIGN KEY (avenue_id) REFERENCES avenues(id) ON DELETE RESTRICT
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- =====
-- 4. FORMULES D'ACQUISITION
-- =====

CREATE TABLE payment_plans (
    id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    code VARCHAR(50) NOT NULL UNIQUE,
    name VARCHAR(150) NOT NULL,
    total_price DECIMAL(12,2) NOT NULL,
    monthly_amount DECIMAL(12,2) NULL,
    duration_months INT UNSIGNED NOT NULL DEFAULT 0,
    frequency ENUM('once', 'monthly') NOT NULL,
    active BOOLEAN NOT NULL DEFAULT TRUE,
    valid_from DATE NULL,
    valid_until DATE NULL,
    description TEXT NULL,
    created_at TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    CONSTRAINT chk_payment_plans_total_price CHECK (total_price >= 0),
    CONSTRAINT chk_payment_plans_monthly_amount CHECK (monthly_amount IS NULL OR monthly_amount >= 0),
    CONSTRAINT chk_payment_plans_dates CHECK (
        valid_until IS NULL OR valid_from IS NULL OR valid_until >= valid_from
    )
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

INSERT INTO payment_plans
    (code, name, total_price, monthly_amount, duration_months, frequency, active)
VALUES
    ('CASH', 'Formule Cash', 2500.00, NULL, 0, 'once', TRUE),
    ('CREDIT_1Y', 'Credit 1 an', 3600.00, 300.00, 12, 'monthly', TRUE),
    ('CREDIT_3Y', 'Credit 3 ans', 6200.00, 175.00, 36, 'monthly', TRUE),
    ('CREDIT_5Y', 'Credit 5 ans', 7500.00, 125.00, 60, 'monthly', TRUE),
    ('CREDIT_10Y', 'Credit 10 ans', 9000.00, 75.00, 120, 'monthly', TRUE);

-- =====
-- 5. SOUSCRIPTIONS
-- Un client peut posséder plusieurs parcelles via plusieurs souscriptions.
-- Une parcelle ne peut avoir qu'une seule souscription active à la fois.
-- =====

CREATE TABLE subscriptions (
    id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    subscription_number VARCHAR(50) NOT NULL UNIQUE,
    customer_id BIGINT UNSIGNED NOT NULL,
    plot_id BIGINT UNSIGNED NOT NULL,
    payment_plan_id BIGINT UNSIGNED NOT NULL,
    subscription_date DATE NOT NULL,
    start_date DATE NOT NULL,
    expected_end_date DATE NULL,

    -- Snapshot contractuel : ces valeurs ne changent pas si la formule commerciale évolue.
    contract_total DECIMAL(12,2) NOT NULL,
    monthly_amount DECIMAL(12,2) NULL,
    duration_months INT UNSIGNED NOT NULL DEFAULT 0,

    amount_paid DECIMAL(12,2) NOT NULL DEFAULT 0,
    balance DECIMAL(12,2) GENERATED ALWAYS AS (contract_total - amount_paid) STORED,

    commercial_status ENUM(
        'draft', 'pending', 'active', 'suspended',
        'cancelled', 'terminated', 'completed'
    )

```

```

) NOT NULL DEFAULT 'active',
financial_status ENUM('unpaid', 'partially_paid', 'paid', 'overdue')
NOT NULL DEFAULT 'unpaid',
administrative_status ENUM('not_started', 'in_progress', 'validated', 'completed')
NOT NULL DEFAULT 'not_started',

-- Astuce MySQL : cette colonne generee permet un index UNIQUE partiel equivalent.
-- Elle vaut plot_id uniquement pour une souscription encore active, sinon NULL.
active_plot_guard BIGINT UNSIGNED GENERATED ALWAYS AS (
CASE
WHEN commercial_status IN ('pending', 'active', 'suspended') THEN plot_id
ELSE NULL
END
) STORED,

created_by BIGINT UNSIGNED NULL,
created_at TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,

UNIQUE KEY uq_active_subscription_per_plot (active_plot_guard),
INDEX idx_subscriptions_customer (customer_id),
INDEX idx_subscriptions_plot (plot_id),
INDEX idx_subscriptions_payment_plan (payment_plan_id),
INDEX idx_subscriptions_financial_status (financial_status),

CONSTRAINT chk_subscriptions_contract_total CHECK (contract_total >= 0),
CONSTRAINT chk_subscriptions_monthly_amount CHECK (monthly_amount IS NULL OR monthly_amount >= 0),
CONSTRAINT chk_subscriptions_amount_paid CHECK (amount_paid >= 0 AND amount_paid <= contract_total),

CONSTRAINT fk_subscriptions_customer
FOREIGN KEY (customer_id) REFERENCES customers(id) ON DELETE RESTRICT,
CONSTRAINT fk_subscriptions_plot
FOREIGN KEY (plot_id) REFERENCES plots(id) ON DELETE RESTRICT,
CONSTRAINT fk_subscriptions_payment_plan
FOREIGN KEY (payment_plan_id) REFERENCES payment_plans(id) ON DELETE RESTRICT,
CONSTRAINT fk_subscriptions_created_by
FOREIGN KEY (created_by) REFERENCES users(id) ON DELETE SET NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- =====
-- 6. CONTRATS
-- =====

CREATE TABLE contracts (
id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
contract_number VARCHAR(50) NOT NULL UNIQUE,
subscription_id BIGINT UNSIGNED NOT NULL UNIQUE,
signed_at DATE NULL,
document_path TEXT NULL,
status ENUM('draft', 'signed', 'cancelled', 'archived') NOT NULL DEFAULT 'draft',
created_at TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
CONSTRAINT fk_contracts_subscription
FOREIGN KEY (subscription_id) REFERENCES subscriptions(id) ON DELETE RESTRICT
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- =====
-- 7. ECHEANCIERS
-- =====

CREATE TABLE installments (
id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
subscription_id BIGINT UNSIGNED NOT NULL,
installment_number INT UNSIGNED NOT NULL,
due_date DATE NOT NULL,
amount_due DECIMAL(12,2) NOT NULL,
amount_paid DECIMAL(12,2) NOT NULL DEFAULT 0,
balance DECIMAL(12,2) GENERATED ALWAYS AS (amount_due - amount_paid) STORED,
paid_at DATETIME NULL,
status ENUM('upcoming', 'due', 'partially_paid', 'paid', 'overdue')
NOT NULL DEFAULT 'upcoming',
created_at TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,

UNIQUE KEY uq_installment_number (subscription_id, installment_number),
INDEX idx_installments_subscription (subscription_id),
INDEX idx_installments_due_date (due_date),
INDEX idx_installments_status (status),

CONSTRAINT chk_installments_number CHECK (installment_number > 0),
CONSTRAINT chk_installments_amount_due CHECK (amount_due >= 0),
CONSTRAINT chk_installments_amount_paid CHECK (amount_paid >= 0 AND amount_paid <= amount_due),
CONSTRAINT fk_installments_subscription
FOREIGN KEY (subscription_id) REFERENCES subscriptions(id) ON DELETE RESTRICT
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- =====
-- 8. PAIEMENTS
-- Paiements partiels et paiements anticipés autorisés.
-- =====

CREATE TABLE payments (
id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
payment_reference VARCHAR(50) NOT NULL UNIQUE,
customer_id BIGINT UNSIGNED NOT NULL,
subscription_id BIGINT UNSIGNED NOT NULL,
payment_date DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
amount DECIMAL(12,2) NOT NULL,
currency CHAR(3) NOT NULL DEFAULT 'USD',

```

```

payment_method ENUM('cash', 'bank_transfer', 'mobile_money', 'card', 'other') NOT NULL,
transaction_reference VARCHAR(150) NULL,
status ENUM('pending', 'validated', 'cancelled', 'reversed') NOT NULL DEFAULT 'validated',
received_by BIGINT UNSIGNED NULL,
notes TEXT NULL,
created_at TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,

INDEX idx_payments_customer (customer_id),
INDEX idx_payments_subscription (subscription_id),
INDEX idx_payments_date (payment_date),
INDEX idx_payments_status (status),

CONSTRAINT chk_payments_amount CHECK (amount > 0),
CONSTRAINT fk_payments_customer
    FOREIGN KEY (customer_id) REFERENCES customers(id) ON DELETE RESTRICT,
CONSTRAINT fk_payments_subscription
    FOREIGN KEY (subscription_id) REFERENCES subscriptions(id) ON DELETE RESTRICT,
CONSTRAINT fk_payments_received_by
    FOREIGN KEY (received_by) REFERENCES users(id) ON DELETE SET NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

```

```

-- =====
-- 9. AFFECTATION DES PAIEMENTS AUX ECHEANCES
-- =====

```

```

CREATE TABLE payment_allocations (
    id BIGINT UNSIGNED AUTO INCREMENT PRIMARY KEY,
    payment_id BIGINT UNSIGNED NOT NULL,
    installment_id BIGINT UNSIGNED NOT NULL,
    amount DECIMAL(12,2) NOT NULL,
    created_at TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP,

    UNIQUE KEY uq_payment_installment (payment_id, installment_id),
    INDEX idx_payment_allocations_payment (payment_id),
    INDEX idx_payment_allocations_installment (installment_id),

    CONSTRAINT chk_payment_allocations_amount CHECK (amount > 0),
    CONSTRAINT fk_payment_allocations_payment
        FOREIGN KEY (payment_id) REFERENCES payments(id) ON DELETE RESTRICT,
    CONSTRAINT fk_payment_allocations_installment
        FOREIGN KEY (installment_id) REFERENCES installments(id) ON DELETE RESTRICT
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

```

```

-- =====
-- 10. RECUS
-- =====

```

```

CREATE TABLE receipts (
    id BIGINT UNSIGNED AUTO INCREMENT PRIMARY KEY,
    receipt_number VARCHAR(50) NOT NULL UNIQUE,
    payment_id BIGINT UNSIGNED NOT NULL UNIQUE,
    customer_id BIGINT UNSIGNED NOT NULL,
    subscription_id BIGINT UNSIGNED NOT NULL,
    amount DECIMAL(12,2) NOT NULL,
    issued_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    pdf_path TEXT NULL,
    verification_code VARCHAR(150) NULL UNIQUE,
    qr_code_value TEXT NULL,
    issued_by BIGINT UNSIGNED NULL,
    status ENUM('valid', 'cancelled') NOT NULL DEFAULT 'valid',
    created_at TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP,

    INDEX idx_receipts_customer (customer_id),
    INDEX idx_receipts_subscription (subscription_id),
    CONSTRAINT chk_receipts_amount CHECK (amount > 0),
    CONSTRAINT fk_receipts_payment
        FOREIGN KEY (payment_id) REFERENCES payments(id) ON DELETE RESTRICT,
    CONSTRAINT fk_receipts_customer
        FOREIGN KEY (customer_id) REFERENCES customers(id) ON DELETE RESTRICT,
    CONSTRAINT fk_receipts_subscription
        FOREIGN KEY (subscription_id) REFERENCES subscriptions(id) ON DELETE RESTRICT,
    CONSTRAINT fk_receipts_issued_by
        FOREIGN KEY (issued_by) REFERENCES users(id) ON DELETE SET NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

```

```

-- =====
-- 11. AUDIT LOGS
-- =====

```

```

CREATE TABLE audit_logs (
    id BIGINT UNSIGNED AUTO INCREMENT PRIMARY KEY,
    user_id BIGINT UNSIGNED NULL,
    action VARCHAR(100) NOT NULL,
    entity_type VARCHAR(100) NOT NULL,
    entity_id BIGINT UNSIGNED NULL,
    old_values JSON NULL,
    new_values JSON NULL,
    ip_address VARCHAR(45) NULL,
    user_agent TEXT NULL,
    created_at TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP,

    INDEX idx_audit_logs_user (user_id),
    INDEX idx_audit_logs_entity (entity_type, entity_id),
    INDEX idx_audit_logs_created_at (created_at),
    CONSTRAINT fk_audit_logs_user
        FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE SET NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

```

```

-- =====
-- 12. SETTINGS
-- =====

CREATE TABLE settings (
    id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    setting_group VARCHAR(100) NOT NULL,
    setting_key VARCHAR(150) NOT NULL,
    value LONGTEXT NULL,
    value_type ENUM('string', 'integer', 'decimal', 'boolean', 'json') NOT NULL DEFAULT 'string',
    is_public BOOLEAN NOT NULL DEFAULT FALSE,
    updated_by BIGINT UNSIGNED NULL,
    created_at TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,

    UNIQUE KEY uq_settings_group_key (setting_group, setting_key),
    CONSTRAINT fk_settings_updated_by
        FOREIGN KEY (updated_by) REFERENCES users(id) ON DELETE SET NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- =====
-- 13. DONNEES DE BASE - ROLES
-- =====

INSERT INTO roles (name, description) VALUES
    ('super_admin', 'Acces complet a la plateforme'),
    ('admin', 'Administration operationnelle'),
    ('commercial', 'Gestion commerciale et clients'),
    ('cashier', 'Gestion des paiements et recus'),
    ('finance_manager', 'Supervision financiere et rapports'),
    ('direction', 'Consultation des indicateurs et rapports'),
    ('customer', 'Acces au portail client');

-- =====
-- 14. PARAMETRES DE BASE
-- =====

INSERT INTO settings
(setting_group, setting_key, value, value_type, is_public)
VALUES
    ('company', 'name', 'MJIC IMMOBILIER SARL', 'string', TRUE),
    ('project', 'name', 'Cite Etoile du Monde', 'string', TRUE),
    ('finance', 'currency', 'USD', 'string', TRUE),
    ('receipt', 'prefix', 'REC', 'string', FALSE),
    ('subscription', 'allow_partial_payment', 'true', 'boolean', FALSE),
    ('subscription', 'allow_advance_payment', 'true', 'boolean', FALSE);

SET FOREIGN_KEY_CHECKS = 1;

-- =====
-- FIN DU SCHEMA INITIAL
-- Important : les regles transactionnelles sensibles (allocation,
-- reversal, recalcul des soldes, generation des echeanciers) doivent
-- etre implementees dans des services Laravel transactionnels et testees.
-- =====

```